

工业边缘计算网关 (Industrial Edge Computing Gateway)

基于 NXP i.MX6ULL 的嵌入式 Linux 驱动全栈项目

SoC: ARM Cortex-A7 @ 792MHz · 内核: Linux 4.1.15 · 架构: ARMv7 (32-bit) 交叉编译: arm-linux-gnueabi-gcc 5.4.0 · 开发板: 正点原子 i.MX6ULL ALPHA

目录

- 1. 项目定位
- 2. 应用场景
- 3. 系统架构
- 4. 硬件平台
- 5. 项目目录结构
- 6. 驱动层详表
- 7. libedge 基础库
- 8. 用户层应用
- 9. 传感器—执行器联动规则
- 10. 总线类型覆盖
- 11. 开发里程碑
- 12. 编译与部署
- 13. 运行指南
- 14. 文档导航
- 15. 技术栈总览
- 16. 许可证

1. 项目定位

本项目从零构建一个**工业边缘计算网关**，覆盖嵌入式 Linux 驱动开发的完整技术栈。以 NXP i.MX6ULL 为平台，依次实现 12 个内核驱动、1 个自研 C 基础库 (libedge)、1 个基于 epoll 的统一监控守护进程，以及完整的测试和文档体系。

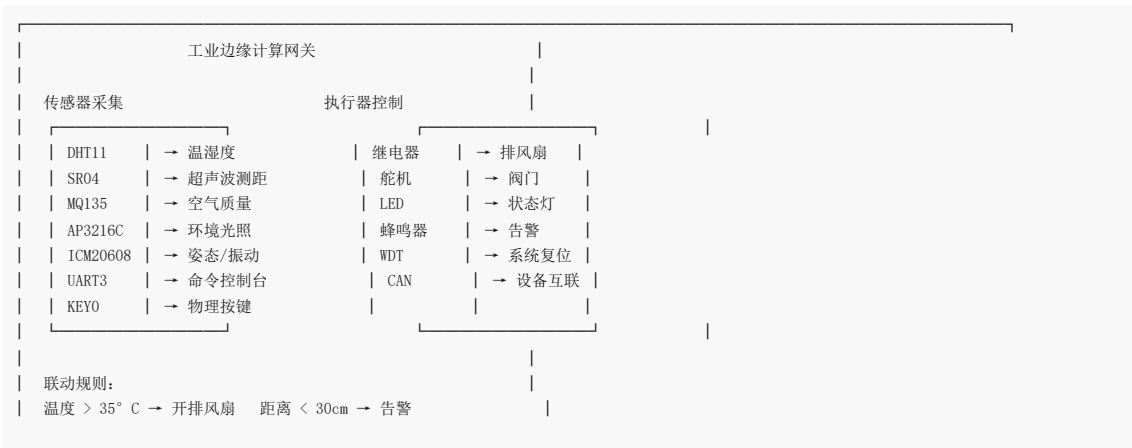
v1.0 (阶段 19, 已完成) : 智能环境监测终端 — 5 个驱动, 基础 **epoll** 框架。 **v2.0** (阶段 1014, 进行中) : 工业边缘计算网关 — 扩展至 12 个驱动 + libedge + 联动控制规则 + 完整测试体系。

核心能力

维度	内容
驱动注册方式	platform_driver / i2c_driver / spi_driver / misc_register / cdev+SocketCAN (6 种)
总线覆盖	GPIO / I2C / SPI / UART / GPIO 单总线 / ADC / PWM / CAN / WDOG (10 种)
IO 模型	阻塞读 / 非阻塞读 / poll+epoll / ioctl / fasync 异步通知 / sysfs
内核机制	wait_queue / completion / kthread / kfifo / delayed_work / timer / mutex / spinlock / atomic
用户空间	epoll 多路复用 / 查表法命令引擎 / 行缓冲拼包 / 信号优雅退出 / 静态库链接
基础库	环形缓冲 / 双向链表 / CRC 校验 / 滑动滤波 / 分级日志 / INI 解析 / 消息队列 / 错误码
测试体系	单元测试 (Unity) → 功能测试 → 集成测试 → 24h 压力测试 (4 层金字塔)

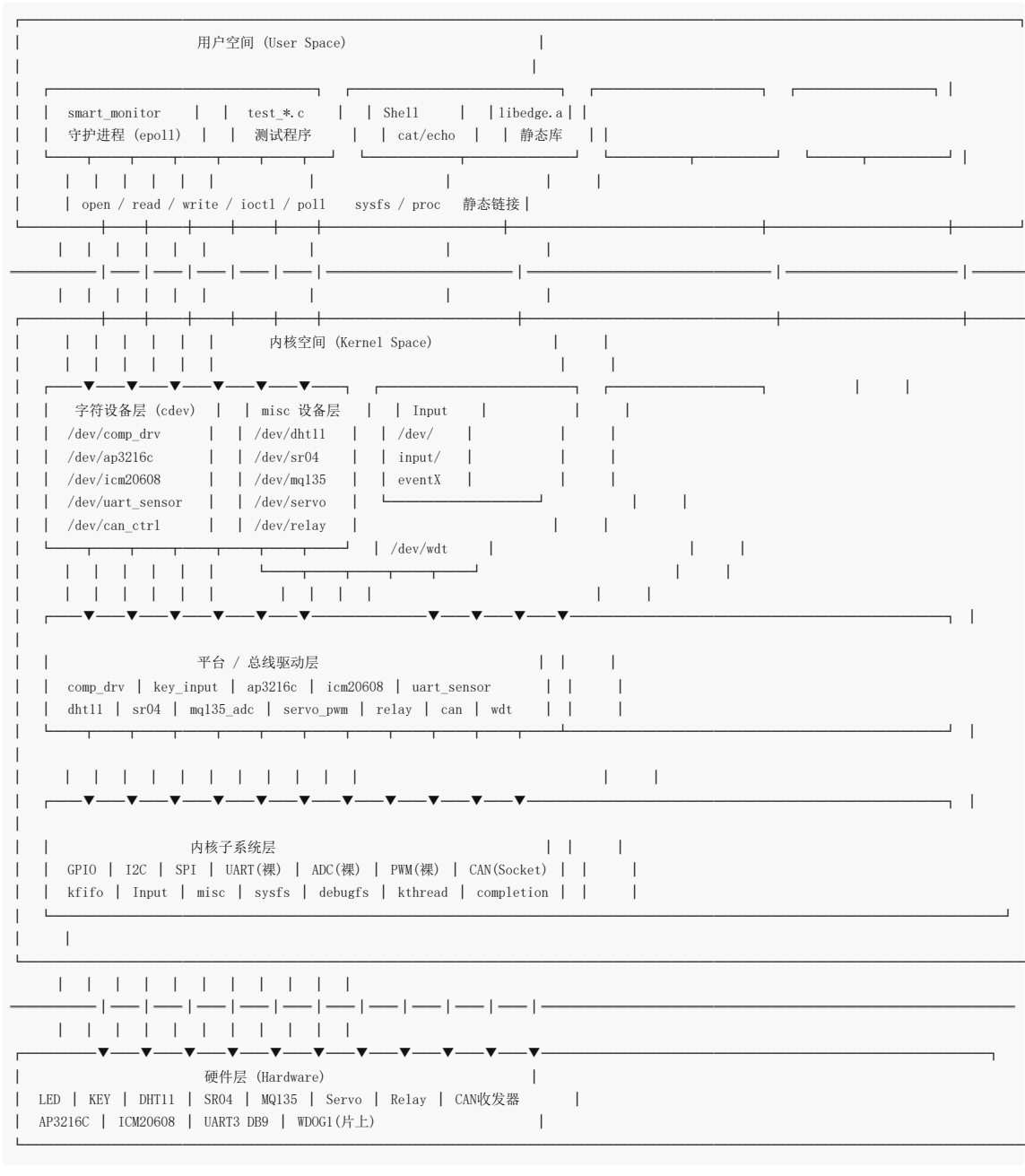
2. 应用场景

以**工厂车间环境监控**为背景:





3. 系统架构



4. 硬件平台

项目	参数
SoC	NXP i.MX6ULL (ARM Cortex-A7 单核, 792MHz)
开发板	正点原子 ALPHA i.MX6ULL (eMMC 版)
RAM	512MB DDR3
存储	8GB eMMC (可 TF 卡扩展)
调试串口	UART1 (Micro USB, CH340 转换)

应用串口	UART3 (DB9 RS232, SP3232 电平转换)
内核版本	Linux 4.1.15 (正点原子官方 BSP)
编译器	arm-linux-gnueabi-hf-gcc 5.4.0

外设全景

#	外设	总线类型	接口/引脚	设备节点	状态
1	LED	GPIO 输出	GPIO1_IO03	/dev/comp_drv	✓
2	KEY0	GPIO 输入 + IRQ	GPIO1_IO18	/dev/input/event0	✓
3	AP3216C	I2C1	地址 0x1E	/dev/ap3216c	✓
4	ICM20608	ECSPi3 CS0	Mode 0, 8MHz	/dev/icm20608	✓
5	UART3	UART 裸寄存器	0x021EC000	/dev/uart_sensor	✓
6	DHT11	GPIO 单总线	待定	/dev/dht11	⬜
7	SR04	GPIO ×2 + IRQ	TRIG + ECHO	/dev/sr04	⬜
8	MQ135	ADC1	待定模拟通道	/dev/mq135	⬜
9	舵机	PWM	待定 PWM 输出	/dev/servo	⬜
10	继电器	GPIO 输出	待定	/dev/relay	⬜
11	CAN	CAN1	TX/RX + 外接收发器	/dev/can_ctrl	⬜
12	WDT	WDOG1 (片上)	—	/dev/wdt_custom	⬜

✓ = v1.0 已完成 ⬜ = v2.0 待实现

5. 项目目录结构

```
imx6ull_project/
├── app/                                # 用户空间程序
│   ├── Makefile                       # 应用构建规则
│   └── smart_monitor.c                # 统一监控守护进程 (~800 行)
├── driver/                             # 内核驱动模块 (Kbuild)
│   ├── Makefile                       # Kbuild 驱动构建
│   ├── comp_drv.c / .h                # LED 控制驱动 (platform + cdev)
│   ├── key_input.c                   # 按键输入驱动 (platform + input)
│   ├── ap3216c.c                     # 光感/接近传感器 (I2C + cdev)
│   ├── icm20608.c / .h               # 6 轴姿态传感器 (SPI + cdev)
│   ├── uart_sensor.c                 # UART 命令控制台 (platform + 裸寄存器)
│   ├── dht11.c                       # DHT11 温湿度 (misc + 单总线)
│   ├── sr04.c                        # SR04 超声波测距 (misc + GPIO IRQ)
│   ├── mq135_adc.c                  # MQ135 空气质量 (misc + 裸 ADC)
│   ├── servo_pwm.c                  # 舵机控制 (misc + 裸 PWM)
│   ├── relay.c                      # 继电器控制 (misc + GPIO)
│   ├── can_drv.c                    # CAN 通信 (cdev + SocketCAN)
│   └── wdt.c                         # 看门狗 (misc + 裸 WDOG)
├── libedge/                            # 自研 C 基础库 (静态链接库)
│   ├── include/                      # 公共头文件
│   │   ├── ringbuf.h                # 泛型环形缓冲
│   │   ├── linked_list.h            # 双向链表 (哨兵节点)
│   │   ├── crc.h                    # CRC 校验 (查表法)
│   │   ├── filter.h                 # 滑动平均 + 中值滤波
│   │   ├── logger.h                 # 分级日志
│   │   ├── ini_parser.h              # INI 配置解析
│   │   ├── msgqueue.h                # POSIX 消息队列封装
│   │   └── edge_error.h              # 统一错误码
│   ├── src/                          # 源文件 (.c)
│   └── tests/                         # 单元测试 (Unity 框架)
└── test/                              # 扩展      # 测试程序
```

├── test_comp_drv.c	# LED 驱动功能测试
├── test_key.c	# 按键驱动测试
├── test_ap3216c.c	# AP3216C 传感器测试
├── test_icm20608.c	# ICM20608 传感器测试
├── test_dht11.c	# DHT11 测试
├── test_sr04.c	# SR04 测试
├── test_mq135.c	# MQ135 测试
├── test_servo.c	# 舵机测试
├── test_relay.c	# 继电器测试
├── test_can.c	# CAN 测试
├── test_wdt.c	# WDT 测试
├── test_ringbuf.c	# libedge 环形缓冲单元测试
├── test_linked_list.c	# libedge 链表单元测试
├── test_crc.c	# libedge CRC 单元测试
├── test_filter.c	# libedge 滤波单元测试
├── test_ini_parser.c	# libedge INI 解析单元测试
├── dts/	# 设备树源文件
├── imx6ull-smart-monitor.dtsi	# 外设节点定义
├── docs/	# 项目文档
├── requirements.md	# 需求规格说明书 (v2.0)
├── architecture.md	# 系统架构设计 (v2.0)
├── api-reference.md	# 12 设备 API 参考手册 (v2.0)
├── hardware.md	# 硬件资源规划 (v2.0)
├── lessons-learned.md	# 全项目知识点汇总 (v2.0, 13 章)
├── execution-flow.md	# v1.0 完整执行流程
├── poll-blocking-wakeup.md	# 内核 poll/wake_up 全链路详解
├── industrial-edge-gateway.md	# v2.0 工业网关设计文档
├── expansion-plan.md	# v2.0 扩展方案
├── expansion-execution-flow.md	# v2.0 扩展执行流程
├── prerequisites-v2.md	# v2.0 前置知识清单
├── phases/	# 9 份阶段开发报告
├── phase1_report.md	# 环境搭建
├── phase2_report.md	# 设备树设计
├── phase3_report.md	# LED CDEV 驱动
├── phase4_report.md	# IO 模型 (阻塞/非阻塞/poll/fasync)
├── phase5_report.md	# 按键 + Input 子系统
├── phase6_report.md	# I2C + SPI 传感器
├── phase7_report.md	# UART 裸寄存器
├── phase8_report.md	# smart_monitor 守护进程
├── phase9_report.md	# 文档交付与发布
├── scripts/	# 自动化脚本
├── build.sh	# 交叉编译 (驱动 + 应用)
├── deploy.sh	# 部署到开发板 (tftp/nfs)
├── test_all.sh	# 运行全部测试
├── checkpatch.sh	# 代码风格检查
├── .vscode/	# VS Code 配置
├── c_cpp_properties.json	# IntelliSense (内核头文件路径)
├── settings.json	# 编辑器配置
├── Makefile	# 顶层构建入口
├── CLAUDE.md	# Claude Code 项目指令
├── README.md	# 本文件

6. 驱动层详表

6.1 v1.0 已有驱动 (5 个, 已完成)

F-D01 — LED 控制驱动 (comp_drv.c)

属性	值
设备节点	/dev/comp_drv
驱动模型	platform_driver + cdev
GPIO	GPIO1_IO03 (低电平有效)

IO 模型	☒ 阻塞读 ☒ 非阻塞读 ☒ poll ☒ ioctl ☒ fasync
特殊机制	sysfs 属性 + debugfs 调试节点 + 内核定时器自动闪烁

操作:

```
read(fd, &status, sizeof(status)); // 阻塞等待状态变化
write(fd, "on", 2); // LED 常亮
write(fd, "off", 3); // LED 熄灭
ioctl(fd, COMP_DRV_START_BLINK); // 启动闪烁
ioctl(fd, COMP_DRV_SET_BLINK_PERIOD, &period_ms); // 设置周期
```

F-D02 — 按键输入驱动 (key_input.c)

属性	值
设备节点	/dev/input/event0
驱动模型	platform_driver + Input 子系统
GPIO	GPIO1_IO18 (低电平有效, 双边沿中断)
中断处理	上半部 ISR → delayed_work 下半部消抖 20ms
键码	KEY_ENTER = 28

```
struct input_event ev;
read(fd, &ev, sizeof(ev));
// ev.type=EV_KEY, ev.code=28, ev.value=0 (释放)/1 (按下)/2 (长按)
```

F-D03 — AP3216C 光感传感器 (ap3216c.c)

属性	值
设备节点	/dev/ap3216c
总线	I2C1, 7 位地址 0x1E
驱动模型	i2c_driver + cdev
通信方式	SMBus 字节读写 + i2c_transfer 突发读 (6 字节)
数据	红外强度 (IR) + 环境光 (ALS) + 接近距离 (PS)

F-D04 — ICM20608 6 轴传感器 (icm20608.c)

属性	值
设备节点	/dev/icm20608
总线	ECSPI3 CS0, SPI Mode 0, max 8MHz
驱动模型	spi_driver + cdev
通信方式	spi_message + spi_transfer 突发读 (14 字节)
IO 模型	read (全数据) + ioctl (分轴读取加速度/角速度/温度)

F-D05 — UART 命令控制台 (uart_sensor.c)

属性	值
设备节点	/dev/uart_sensor
基地址	0x021EC000 (UART3)
驱动模型	platform_driver + cdev + 裸寄存器 ioremap
通信参数	115200-8-N-1, DB9 RS232
IO 模型	☒ 阻塞读 ☒ 非阻塞读 ☒ poll
缓冲	kfifo 环形缓冲 (RX 中断 → kfifo_in → wake_up)

6.2 v2.0 新增驱动 (7 个, 待实现)

F-D06 — DHT11 温湿度 (dht11.c)

属性	值
驱动模型	misc_register (~350 行)
总线	GPIO 单总线 (1-Wire)
协议	主机拉低 18ms → 读 40bit (湿度 + 温度 + 校验)
关键技术	ktime 测 μ s 级脉冲宽度、local_irq_save 保护时序、sysfs 属性
采样间隔	≥ 2 s

```
uint8_t data[4];
read(fd, data, 4); // data[0]=湿度, data[2]=温度
cat /sys/class/misc/dht11/temperature → "27.3"
cat /sys/class/misc/dht11/humidity → "65.0"
```

F-D07 — SR04 超声波测距 (sr04.c)

属性	值
驱动模型	misc_register (~200 行)
总线	GPIO ×2 (TRIG 输出 + ECHO 输入, 双边沿中断)
关键技术	completion 同步、ktime 脉冲测量、1s 超时保护
量程	2cm ~ 400cm

```
uint32_t distance_cm;
read(fd, &distance_cm, sizeof(distance_cm)); // 阻塞等待回波
cat /sys/class/misc/sr04/distance → "125"
```

F-D08 — MQ135 空气质量 (mq135_adc.c)

属性	值
驱动模型	misc_register (~250 行)
总线	ADC1 裸寄存器 (ioremap 0x02198000)
关键技术	kthread 每 2s 周期采样、readl 读 ADC 结果寄存器、滑动平均滤波
sysfs	/sys/class/misc/mq135/raw + /sys/class/misc/mq135/quality

F-D09 — 舵机 PWM 控制 (servo_pwm.c)

属性	值
驱动模型	misc_register (~250 行)
总线	PWM 裸寄存器 (ioremap 0x02080000)
参数	周期 20ms (50Hz) , 占空比 0.5ms 2.5ms → 0°180°
接口	write 设角度, read 读当前角度, sysfs 属性

F-D10 — 继电器控制 (relay.c)

属性	值
驱动模型	misc_register (~120 行)
总线	GPIO 输出 (gpiod API)
接口	write("on"/"off") 控制吸合/断开, read 读状态
应用	控制排风扇 / 加热器 / 照明

F-D11 — CAN 总线通信 (can_drv.c)

属性	值
驱动模型	cdev + SocketCAN (~300 行)
协议	CAN 2.0B (11/29-bit ID, 0~8 字节数据)
接口	read/write can_frame 结构体, poll 支持, ioctl 设波特率/过滤器

硬件	CAN1 控制器 + 外接 TJA1050 收发器
----	---------------------------

```
struct can_frame frame = { .can_id = 0x123, .can_dlc = 2 };
frame.data[0] = 0xAB; frame.data[1] = 0xCD;
write(fd, &frame, sizeof(frame));
```

F-D12 — 看门狗 (wdt.c)

属性	值
驱动模型	misc_register (~120 行)
总线	WDOG1 裸寄存器 (ioremap 0x020BC000)
机制	超时自动复位, 喂狗序列 0x5555 → 0xAAAA
超时范围	0.5s ~ 128s (默认 10s)
ioctl	WDT_SET_TIMEOUT / WDT_GET_TIMELEFT

7. libedge 基础库

自研 C 基础库, 独立编译为 libedge.a , 被应用层静态链接。

#	模块	文件	关键数据结构 / 算法	复杂度
L-01	环形缓冲	ringbuf.h/.c	泛型宏实现, 类型安全, O(1) push/pop	O(1)
L-02	双向链表	linked_list.h/.c	哨兵节点消除 NULL 检查, 归并排序	O(1) 增删 / O(n log n) 排序
L-03	CRC 校验	crc.h/.c	查表法, CRC8 / CRC16-Modbus (多项式 0x8005) / CRC32	O(n)
L-04	滑动滤波	filter.h/.c	滑动平均 (维护 sum, O(1) 更新) + 中值滤波	O(n) / O(n log n)
L-05	分级日志	logger.h/.c	ERROR/WARN/INFO/DEBUG 四级, 时间戳, stdout+文件输出	—
L-06	INI 解析	ini_parser.h/.c	逐行状态机, 支持 [section] / key=value / # 注释	O(n)
L-07	消息队列	msgqueue.h/.c	POSIX mq_open/send/receive 封装	—
L-08	错误码	edge_error.h/.c	统一错误码枚举 + 查表法字符串描述	O(1)

每个模块配 Unity 单元测试, 覆盖率 > 80%。CRC 模块含标准测试向量验证。

8. 用户层应用

smart_monitor — 统一监控守护进程

属性	值
文件	app/smart_monitor.c (~800 行 → v2.0 ~900 行)
IO 模型	epoll 多路复用 (v1.0: 6 fd → v2.0: 13 fd)
命令引擎	查表法 (struct { name, handler }[]), O(1) 分发
退出方式	SIGINT/SIGTERM 信号只置标志位, 主循环判断退出

命令行参数

```
./smart_monitor [选项]
--no-log      不记录日志
--interval N  传感器轮询间隔 (秒, 默认 2)
--help       显示帮助
```

键盘交互

按键	功能
q / ESC	退出程序
s	显示系统运行状态
l / 2	LED 开 / 关
h	显示帮助

UART 命令集 (v2.0 扩展)

命令	格式	功能
STATUS	STATUS	系统运行时间和传感器在线数
SENSOR	SENSOR	全部传感器当前值
LED	LED ON / LED OFF / LED BLINK=500	LED 控制
RELAY	RELAY ON / RELAY OFF	继电器控制 
SERVO	SERVO <0~180>	舵机角度 
CAN	CAN SEND <id>:<data>	CAN 帧发送 
HELP	HELP	列出所有命令
RESET	RESET	重置所有设备到默认状态

核心事件循环

```
smart_monitor 启动
├── 解析命令行参数
├── setup_epoll()
│   ├── epoll_create1(0) → epfd
│   └── 循环: open 设备 → fcntl(0_NONBLOCK) → epoll_ctl(ADD, fd, EPOLLIN)
└── run_event_loop()
    ├── 设置定时器 (timerfd_create, 每 N 秒触发)
    ├── 注册信号处理 (SIGINT/SIGTERM → g_running = 0)
    │   └── while (g_running):
    │       epoll_wait(epfd, events, 13, timeout)
    │       ├── stdin 可读 (fd[0])      → 处理键盘输入
    │       ├── timer 触发 (fd[1])      → 轮询传感器 (ap3216c/icm20608/mq135)
    │       ├── LED 可读 (fd[2])        → 打印 LED 状态
    │       ├── KEY 可读 (fd[3])        → 处理按键事件
    │       ├── UART 可读 (fd[4])       → 行缓冲拼包 → 查表发命令分发
    │       ├── AP3216C 可读 (fd[5])    → 打印传感器数据 + 联动判断
    │       ├── ICM20608 可读 (fd[6])   → 打印加速度/角速度
    │       ├── DHT11 可读 (fd[7])  → 温湿度联动判断
    │       ├── SR04 可读 (fd[8])  → 距离联动判断
    │       ├── MQ135 可读 (fd[9])  → 空气质量联动判断
    │       ├── Servo 可读 (fd[10])  → 舵机状态
    │       ├── Relay 可读 (fd[11])  → 继电器状态
    │       └── CAN 可读 (fd[12])  → CAN 帧处理
```

9. 传感器—执行器联动规则

#	规则	触发条件	执行动作
R-01	高温排风	DHT11 温度 > 35°C (连续 3 次)	继电器吸合 (排风扇) + LED 闪烁
R-02	高温恢复	DHT11 温度 < 30°C (连续 3 次)	继电器断开 + LED 常亮
R-03	空气污染	MQ135 ≥ 4 级 (中度污染)	蜂鸣器告警 + 继电器吸合
R-04	人员接近	SR04 距离 < 30cm (连续 2 次)	蜂鸣器告警
R-05	振动异常	ICM20608 加速度幅值 > 阈值	LED 告警
R-06	系统死机	WDT 超时	系统自动复位

10. 总线类型覆盖

v2.0 完成后将覆盖 **10 种总线 / 外设类型**:

总线	英文	线数	速度	驱动	版本
GPIO 输出	GPIO Output	1	—	comp_drv, relay	v1.0 / v2.0

GPIO 输入 + IRQ	GPIO Input + IRQ	1	—	key_input, sr04	v1.0 / v2.0
GPIO 单总线	1-Wire	1	~1 Kbps	dht11	v2.0
I²C	I2C	2	100 KHz	ap3216c	v1.0
SPI	SPI	4	8 MHz	icm20608	v1.0
UART（裸寄存器）	UART (bare-metal)	2	115200 bps	uart_sensor	v1.0
ADC（裸寄存器）	ADC (bare-metal)	1 (模拟)	~1 Msps	mq135_adc	v2.0
PWM（裸寄存器）	PWM (bare-metal)	1	50 Hz	servo_pwm	v2.0
CAN 2.0B	CAN	2 (差分)	500 Kbps	can_drv	v2.0
WDOG（裸寄存器）	Watchdog	片上	—	wdt	v2.0

11. 开发里程碑

v1.0 — 智能环境监测终端（阶段 1~9, ✔ 已完成）

阶段	内容	关键技术
1	项目立项与环境搭建	交叉编译工具链，内核源码树
2	设备树设计	compatible / reg / interrupts / gpios
3	LED CDEV 驱动	cdev_init / cdev_add / class_create / device_create
4	IO 模型（并发）	阻塞/非阻塞/poll/fasync/wait_queue/mutex/atomic
5	按键输入驱动	GPIO IRQ + Input 子系统 + delayed_work 消抖
6	I2C + SPI 传感器	i2c_driver / spi_driver / SMBus / spi_message
7	UART 裸寄存器驱动	ioremap + readb/writeb + kfifo + RX 中断
8	统一监控程序	epoll + 查表命令引擎 + 信号优雅退出
9	文档交付与发布	需求/架构/API/硬件文档 + 9 份阶段报告

v2.0 — 工业边缘计算网关（阶段 10~14, ▢ 待实现）

阶段	内容	预计产出
10	libedge 基础库	8 个模块 + 5 个单元测试
11	简单驱动	relay + wdt + servo（3 个 misc 驱动 + 功能测试）
12	中等驱动	dht11 + sr04 + mq135 + can（4 个驱动 + 功能测试）
13	smart_monitor 扩展 + 联动	12 设备集成 + 6 条联动规则
14	文档 + 测试 + 发布	完整测试报告 + v2.0.0 发布

12. 编译与部署

环境要求

组件	要求
宿主机	Ubuntu 16.04（192.168.80.106, 用户 yang）
Windows	通过 Samba 编辑代码, SSH 到 Linux 编译
交叉编译器	arm-linux-gnueabihf-gcc 5.4.0
内核源码树	~/linux/alentek_uboot/linux/linux-imx-rel_imx_4.1.15_2.1.0_ga_alentek

编译命令

```
# SSH 到 Linux 编译服务器
ssh yang@192.168.80.106

cd ~/linux/imx6ull_project

# 编译全部（驱动 + 应用 + libedge）
```

```
make all

# 仅编译内核驱动
make modules

# 仅编译应用 + libedge
make apps

# 清理编译产物
make clean

# 亦可用脚本
./scripts/build.sh      # 完整编译
./scripts/checkpatch.sh # 代码风格检查
```

部署

```
# 将 .ko 文件传输到开发板 (tftp/nfs)
./scripts/deploy.sh

# 在开发板上
insmod /path/to/comp_drv.ko      # 加载 LED 驱动
insmod /path/to/key_input.ko     # 加载按键驱动
insmod /path/to/ap3216c.ko       # 加载光感驱动
insmod /path/to/icm20608.ko      # 加载 6 轴驱动
insmod /path/to/uart_sensor.ko   # 加载 UART 驱动

# 运行监控程序
./smart_monitor --interval 2
```

13. 运行指南

交互示例

```
$ ./smart_monitor --interval 2

=====

Smart Environment Monitor v2.0
Industrial Edge Computing Gateway
=====

[INFO] 12 devices detected
[INFO] Event loop started (timer interval: 2s)

s                               ← 按 s 键
[STATUS] Uptime: 127 sec, Sensors: 12/12 online

→ PC 通过 UART3 发送 "SENSOR"
[SENSOR] Temperature: 27.3° C | Humidity: 65.0% | Light: 340 lux
[SENSOR] Distance: 125cm | Air Quality: good (ADC: 412)
[SENSOR] Accel: X=0.02 Y=-0.01 Z=1.00g | Gyro: X=0.1 Y=0.2 Z=0.0 dps

→ PC 通过 UART3 发送 "RELAY ON"
[ACTION] Relay: ON (exhaust fan started)    ← 温度太高自动触发
```

联动触发日志

```
[2026-06-10 14:32:15] [WARN] Temperature 36.1° C > 35° C (count: 3/3)
[2026-06-10 14:32:15] [ACTION] Rule R-01 triggered: Relay ON, LED blinking
[2026-06-10 14:32:20] [INFO] Temperature 29.5° C < 30° C (count: 3/3)
[2026-06-10 14:32:20] [ACTION] Rule R-02 triggered: Relay OFF, LED on

[2026-06-10 14:35:42] [WARN] Distance 25cm < 30cm (count: 2/2)
[2026-06-10 14:35:42] [ACTION] Rule R-04 triggered: Buzzer ON
```

14. 文档导航

核心文档

文档	文件	说明
项目总览	README.md	本文件
需求规格	docs/requirements.md	v2.0 功能需求、联动规则、测试体系、里程碑
架构设计	docs/architecture.md	分层架构图、IO 矩阵、驱动注册方式对比、决策记录
API 参考	docs/api-reference.md	12 个设备完整 API (read/write/ioctl/poll/sysfs)
硬件资源	docs/hardware.md	引脚分配、内存映射、时序参数、电源需求
知识点汇总	docs/lessons-learned.md	13 章、50+ 知识点，含踩坑记录
前置知识	docs/prerequisites-v2.md	v2.0 开发前需掌握的 24 个知识点

深入学习

文档	说明
docs/execution-flow.md	v1.0 从开机到关机的完整执行路径
docs/poll-blocking-wakeup.md	内核 poll/wake_up/epoll 全链路深度解析 (~1138 行)
docs/industrial-edge-gateway.md	v2.0 工业网关架构设计 (12 驱动 + 8 库模块)
docs/expansion-plan.md	v2.0 扩展方案详细规划
docs/expansion-execution-flow.md	v2.0 扩展执行流程
docs/phases/phase1`9_report.md	9 份阶段开发报告 (每阶段一篇)

15. 技术栈总览

内核层

技术点	具体内容
驱动注册	platform_driver / i2c_driver / spi_driver / misc_register / cdev+SocketCAN
并发同步	mutex / spinlock / atomic / wait_queue / completion / kthread
中断处理	ISR 上半部 / delayed_work 下半部 / kthread 周期采集 / completion 同步
时间	ktime 纳秒精度 / jiffies / timer_list 内核定时器
内存映射	ioremap → readl/writel (UART / ADC / PWM / WDOG 裸寄存器)
字符设备	cdev + misc 两种注册方式 / ioctl / sysfs / debugfs / fasync
内核缓冲	kfifo 环形缓冲 (中断安全)
总线	I2C (SMBus + i2c_transfer) / SPI (spi_sync + spi_message) / UART (裸寄存器)
SocketCAN	PF_CAN / can_frame / CAN 过滤器 / ioctl 波特率设置

用户层

技术点	具体内容
IO 多路复用	epoll_create1 / epoll_ctl / epoll_wait (O(1) 事件分发)
Input 子系统	struct input_event (EV_KEY / KEY_ENTER)
信号处理	signal() 只置标志位 + 主循环检查退出 (优雅退出)
设计模式	查表法命令引擎 / 行缓冲拼包 / Toggle 状态机 / 信号标志退出
应用定时	timerfd_create + epoll 集成 (统一事件源)
联动控制	全局状态共享 + 连续 N 次去抖触发

libedge 库

技术点	具体内容
数据结构	泛型环形缓冲 (宏) / 双向链表 (哨兵节点)
算法	CRC 查表法 (3 种) / 归并排序 / 滑动平均 O(1) 优化 / 中值滤波

工具	分级日志 (va_list 可变参数) / INI 状态机解析 / POSIX 消息队列封装
测试	Unity 框架 / 标准测试向量验证 / 边界条件覆盖

16. 许可证

GPL v2

文档版本: v2.0 | 更新日期: 2026-06-10